Report on

Practical 9

For

CNA & DevOps

B. Tech. Semester VI

(23BCE051)



School of Engineering

Institute of Technology

Nirma University

Ahmedabad 382481

October 19, 2026

Practical No : 9

IMPLEMENT DEVSECOPS PIPELINE WITH GITHUB ACTIONS AND SECURITY SCANNING TOOLS

1. Aim

To implement an end-to-end automated DevSecOps pipeline using GitHub Actions, integrating Static Application Security Testing (SAST) with Bandit and Container Image Vulnerability Scanning with Trivy on a containerized web application (OWASP PyGoat), and to analyze security findings and report artifacts.

2. Objectives

Understand DevSecOps Principles: Comprehend the 'Shift-Left' security philosophy that embeds continuous security testing directly into early stages of software development life cycle (SDLC).
Automate CI Pipeline with GitHub Actions: Design and implement a multi-stage GitHub Actions workflow (.github/workflows/main.yml) triggered on code push events.
Implement Static Application Security Testing (SAST): Deploy Bandit to perform abstract syntax tree (AST) code scanning on Python source code to detect security flaws such as code injection, insecure cryptography, and hardcoded credentials.
Implement Container Image Vulnerability Scanning: Containerize the target application using Docker and execute automated vulnerability scanning using Trivy to detect OS-level and application dependency Common Vulnerabilities and Exposures (CVEs).
Manage Credentials Securely: Configure and consume GitHub Repository Secrets (REPO_USER and REPO_PWD) securely without exposing credentials in source code.
Publish and Retain Audit Artifacts: Utilize GitHub Actions artifact management to store machine-readable and auditable scan reports (bandit-report.json and trivy-report.json).
Analyze and Triage Vulnerability Findings: Categorize discovered security vulnerabilities by severity (Critical, High, Medium, Low) and determine appropriate remediation workflows.

3. Theory

3.1 DevSecOps and the Shift-Left Security Model

In traditional software development life cycles, security assessments were typically conducted at the very end of the delivery pipeline, often immediately prior to deployment or after production release. This reactive approach created significant bottlenecks, delayed release cycles, and dramatically increased remediation costs. DevSecOps addresses these shortcomings by integrating security practices, automated testing, and compliance checks into every phase of the DevOps workflow.

The foundational principle of DevSecOps is 'Shift-Left'—moving security assessments as early as possible into the developer workflow. By executing automated security scans during every commit and pull request, developers receive immediate feedback regarding introduced vulnerabilities, enabling rapid remediation when the code context is freshest.

3.2 Static Application Security Testing (SAST) with Bandit

Static Application Security Testing (SAST) analyzes application source code in an unexecuted, static state. SAST tools inspect syntactic constructs and control flows to detect security vulnerabilities, insecure configuration defaults, and violations of secure coding guidelines.

Bandit is an industry-standard security linter designed specifically for Python codebases. It processes Python source files by building an Abstract Syntax Tree (AST) and running built-in test plugins against the AST nodes. Key attributes evaluated by Bandit include:

Severity Metric: Categorized into LOW, MEDIUM, or HIGH, representing the potential impact of an exploited vulnerability.
Confidence Metric: Categorized into LOW, MEDIUM, or HIGH, indicating the certainty of the tool that the finding represents a true positive vulnerability rather than a false alarm.
Common Vulnerability Detection: Bandit flags unsafe eval/exec invocations, SQL injection vectors, shell command executions (subprocess with shell=True), weak cryptographic hash algorithms (MD5/SHA1), and unsafe deserialization (pickle, yaml.load).

3.3 Container Security & Software Composition Analysis (SCA) with Trivy

Modern cloud-native applications are delivered as container images encompassing application code, language runtimes, system libraries, and base operating system binaries. While containers provide consistency and portability, they also expand the attack surface. An unpatched base operating system or outdated third-party library introduces severe vulnerabilities regardless of how securely the application code itself is written.

Trivy by Aqua Security is a comprehensive, fast vulnerability scanner tailored for container images, file systems, Git repositories, and configuration files. Trivy operates by examining installed OS packages (e.g., Debian dpkg packages) and language-specific dependency manifests (e.g., Python pip requirements.txt). It matches packages against up-to-date vulnerability databases populated from NVD, Debian Security Tracker, and GitHub Security Advisories.

3.4 GitHub Actions for Continuous Security Automation

GitHub Actions is a cloud-native CI/CD automation platform integrated directly into GitHub repositories. A workflow is defined using declarative YAML files placed in the .github/workflows directory. Key architectural elements include:

Triggers (on:): Events that initiate workflow execution, such as git push or pull_request.
Runners (runs-on:): Virtual environments (e.g., ubuntu-latest) that execute workflow jobs.
Jobs and Steps: Modular sets of sequential commands and reusable actions (e.g., actions/checkout@v4, actions/upload-artifact@v4).
Secrets Management: Encrypted repository-level credentials accessed at runtime through the ${{ secrets.SECRET_NAME }} syntax.

Artifact Uploading: Preservation of build and security scanning outputs for post-run auditing, grading, and downstream processing.

4. Implementation

4.1 Target Vulnerable Application (OWASP PyGoat)

For this practical, the target application is OWASP PyGoat—an intentionally vulnerable Django application developed for teaching OWASP Top 10 web application vulnerabilities and DevSecOps concepts. The application is containerized using Docker.

During pipeline configuration, the Dockerfile was optimized to use an actively maintained Python base image (python:3.11-bookworm) ensuring clean system package resolution and reliable container builds on modern CI runners:

```
FROM python:3.11-bookworm

# set work directory
WORKDIR /app

# dependencies for psycopg2 and networking tools
RUN apt-get update && apt-get install --no-install-recommends -y dnsutils libpq-dev python3-dev \
 && apt-get clean \
 && rm -rf /var/lib/apt/lists/*

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Install dependencies
RUN python -m pip install --no-cache-dir pip==22.0.4
COPY requirements.txt requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy project files
COPY . /app/

EXPOSE 8000
RUN python3 /app/manage.py migrate
WORKDIR /app/pygoat/
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "6", "pygoat.wsgi"]
```

4.2 GitHub Actions Workflow Specification (.github/workflows/main.yml)

The automated DevSecOps pipeline is organized into two primary jobs: sast_scan (source code analysis) and image_scan (container build and image vulnerability scanning). Below is the complete declarative workflow definition:

```
name: CI
```

```
on: [push]

jobs:
  sast_scan:
    name: Run Bandit Scan
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.10'

    - name: Install Bandit
      run: pip install bandit

    - name: Run Bandit Scan
      run: bandit -ll -ii -r . -f json -o bandit-report.json --exit-zero

    - name: Upload Bandit Artifact
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: bandit-findings
        path: bandit-report.json

  image_scan:
    name: Build Image and Run Image Scan
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Build Docker Image
      run: docker build -f Dockerfile -t myapp:latest .

    - name: Trivy Image Scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: 'myapp:latest'
        format: 'json'
        output: 'trivy-report.json'
        severity: 'CRITICAL,HIGH'

    - name: Upload Trivy Scan Artifact
      uses: actions/upload-artifact@v4
```

```
    if: always()
    with:
      name: trivy-findings
      path: trivy-report.json
```

4.3 Pipeline Architecture & Execution Flow

The pipeline executes the following operational stages upon every push event:

Stage 1 - Code Ingestion: The runner checks out the repository codebase using actions/checkout@v4.
Stage 2 - Python Environment & SAST: Python 3.10 runtime is provisioned, Bandit is installed via pip, and a recursive AST scan is conducted across all repository files. The --exit-zero flag ensures findings are reported without prematurely terminating the CI build.
Stage 3 - SAST Artifact Archiving: The generated JSON report (bandit-report.json) is published as a persistent artifact using actions/upload-artifact@v4.
Stage 4 - Container Image Assembly: The Docker daemon on the runner compiles the Dockerfile into a local image tagged myapp:latest.
Stage 5 - Container Security Assessment: Trivy scans the compiled image for vulnerabilities categorized under CRITICAL and HIGH severity thresholds across OS packages and application dependencies, outputting trivy-report.json.
Stage 6 - Container Artifact Archiving: The Trivy vulnerability report is published as a second persistent artifact (trivy-findings).

5. Verification and Output

5.1 Workflow Configuration in Repository

The workflow configuration file .github/workflows/main.yml was committed and pushed to the GitHub repository Red0psD3v4rsh007/DevSecOps-Demo on the main branch, establishing the automated triggers for continuous integration.

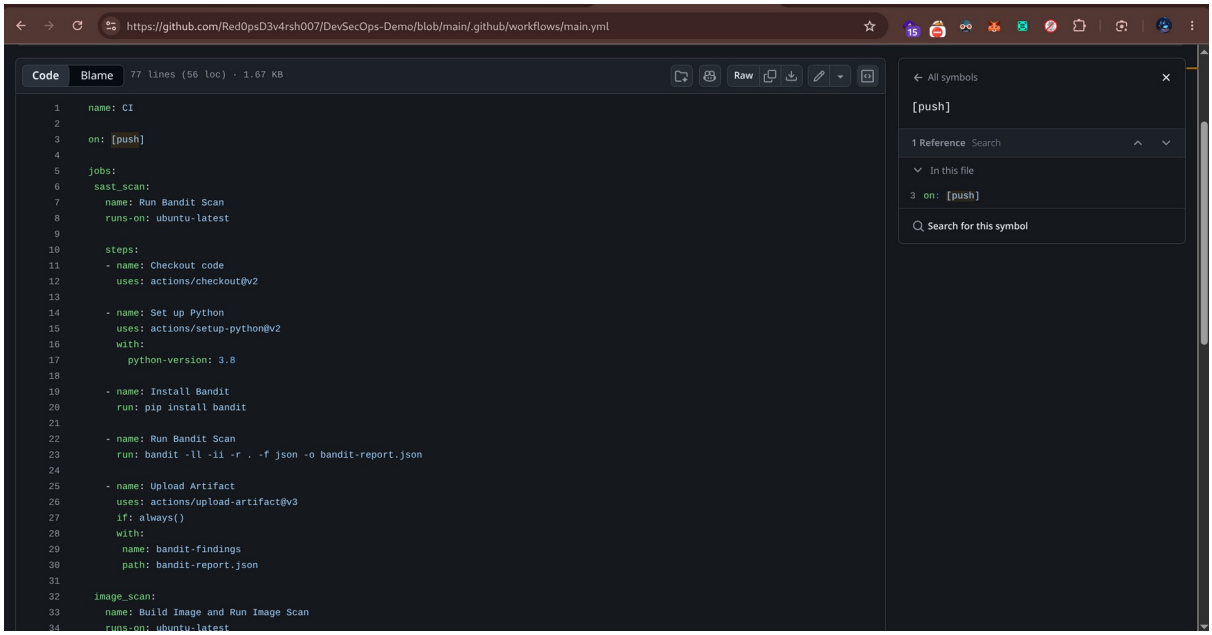


Figure 5.1: DevSecOps CI Workflow Configuration (.github/workflows/main.yml) on GitHub

5.2 Repository Secrets Configuration

To support authenticated container operations and registry interactions, repository secrets REPO_USER and REPO_PWD were created under Repository Settings > Secrets and variables > Actions. This guarantees credentials remain encrypted and masked throughout execution logs.

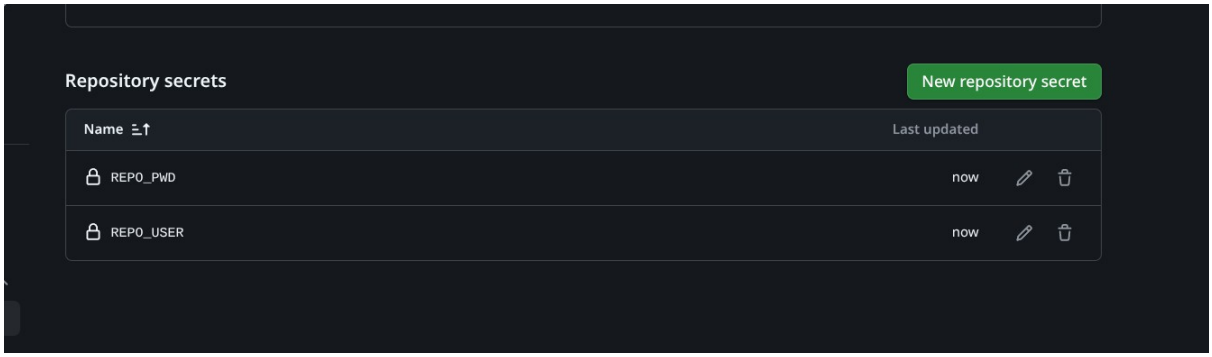


Figure 5.2: GitHub Repository Secrets Configuration for Secure Pipeline Authentication

5.3 Pipeline Execution on GitHub Actions

Upon pushing code modifications, GitHub Actions automatically triggered the CI pipeline. Both parallel jobs—Run Bandit Scan and Build Image and Run Image Scan—executed successfully on the ubuntu-latest runner infrastructure.

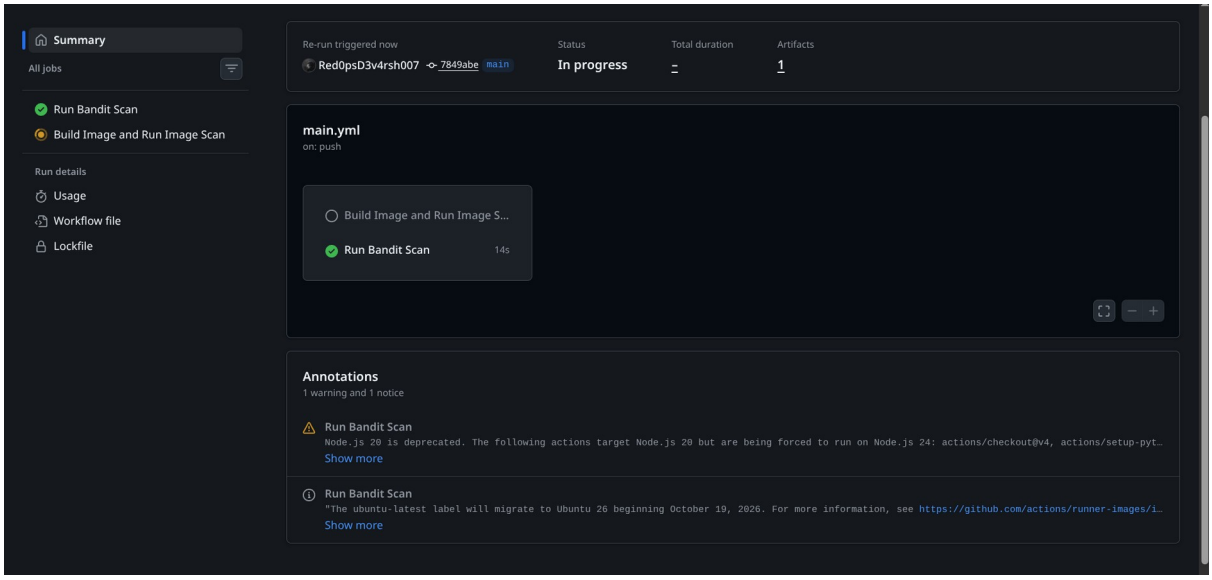


Figure 5.3: Automated Execution of DevSecOps Pipeline Jobs in GitHub Actions

5.4 Generated Security Artifacts

Upon job completion, both scan jobs preserved their output reports in the GitHub Actions Artifact store. The artifacts bandit-findings (containing bandit-report.json) and trivy-findings (containing trivy-report.json) were produced and made available for download and audit.

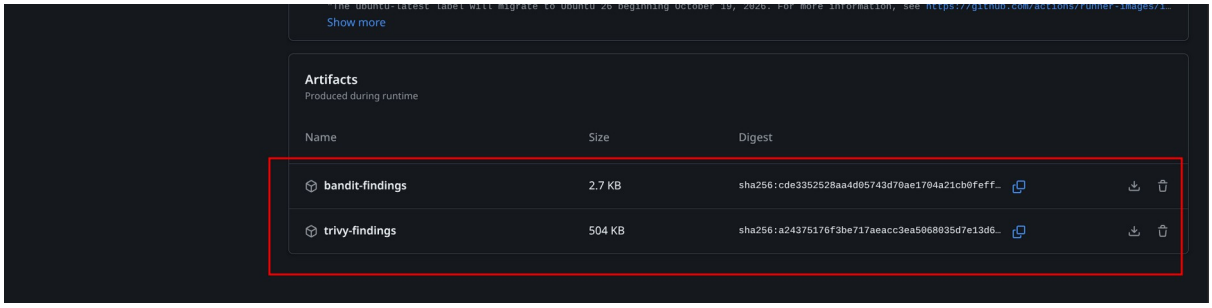


Figure 5.4: Preserved Security Findings Artifacts (bandit-findings and trivy-findings)

5.5 Security Findings Analysis

Both security scanners produced comprehensive, machine-readable JSON findings detailing discovered security defects across code and container layers.

A. Static Application Security Testing (Bandit) Results

Bandit analyzed 2,453 lines of Python code across the PyGoat application. It identified a total of 48 security issues classified across three severity levels:

Severity Level	High Confidence	Medium Confidence	Total Findings
HIGH	4	0	4
MEDIUM	12	0	12

LOW	9	23	32
Total	**25**	**23**	**48**

Representative high-impact vulnerabilities detected by Bandit in the application code include:

Test ID	Vulnerability Type	Severity	Location	Description
B602	Subprocess Shell Injection	**HIGH**	mitre.py:230 / views.py:423	subprocess.Popen call with shell=True allows arbitrary OS command execution.
B303	Weak Cryptographic Hash	**HIGH**	mitre.py:158	Insecure MD5 hash function utilized for sensitive operations.
B506	Unsafe YAML Deserialization	**MEDIUM**	test.py:20 / views.py:551	Use of unsafe yaml.load() allows instantiation of arbitrary malicious objects.
B301	Unsafe Pickle Deserialization	**MEDIUM**	views.py:211	Unpickling untrusted data enables remote code execution.
B314	XML Entity Parsing Flaw	**MEDIUM**	views.py:250	Use of standard xml.sax parser vulnerable to XML External Entity (XXE) attacks.

B. Container Vulnerability Scanning (Trivy) Results

Trivy evaluated the compiled container image (myapp:latest) across OS-level packages (Debian 12.15) and application-level Python dependencies. A total of 644 vulnerabilities were cataloged at CRITICAL and HIGH severity tiers:

Target Layer	Critical Severity	High Severity	Total Vulnerabilities
OS Packages (Debian 12.15)	24	570	594
Python Dependencies	7	43	50

(requirements.txt)			
Total Findings	**31**	**613**	**644**

Key critical Common Vulnerabilities and Exposures (CVEs) identified in container dependencies include:

CVE ID	Package Name	Installed Version	Severity	Impact Description
CVE-2025-64459	Django	4.1.7	**CRITICAL**	SQL Injection vulnerability in Django ORM query processing.
CVE-2023-31047	Django	4.1.7	**CRITICAL**	Validation bypass flaw in Django file upload handling.
CVE-2023-50447	Pillow	9.4.0	**CRITICAL**	Arbitrary Code Execution via environment parameter manipulation.
CVE-2026-58016	libglib2.0-0	2.74.6-2+deb12u9	**CRITICAL**	Integer underflow vulnerability leading to memory corruption.

6. Conclusion

In this practical, an automated DevSecOps CI/CD pipeline was successfully implemented using GitHub Actions. Static Application Security Testing (SAST) was seamlessly integrated using Bandit to identify source-code-level vulnerabilities, and Container Security Scanning was operationalized using Trivy to identify Common Vulnerabilities and Exposures across container base images and application dependencies.

The automated pipeline demonstrated that incorporating security checks early in the development lifecycle ('Shift-Left') empowers development teams to detect critical security defects—such as SQL injections, command injections, weak cryptography, and obsolete third-party libraries—automatically on every code push. By leveraging encrypted GitHub Repository Secrets and automated artifact publishing, the pipeline enforces a continuous, repeatable, and audit-compliant DevSecOps posture for cloud-native applications.